

Predictive Attack Path and Time Intelligence System

Using machine learning

Group Members:	Group number
1.J.Venkata Siva Manikanta (cys24022)	21
2.V.Raj Rizwan (cys24047)	21

Proposed Methodology

The framework to forecast attack paths and threat intelligence analysis via Machine Learning algorithm involves systematic methodology in order to achieve high efficiency and effectiveness in analyzing attack paths and threat intelligence. Raw network traffic data would be transformed into information that would assist in accurate classification of network flows as well as forecasting of the attack path. Systematic process includes many processes starting from collecting datasets to the evaluation of machine learning model performance in classifying malicious and benign flows of network.

Dataset Collection

The first methodology involved obtaining a suitable dataset since the performance of the Machine Learning model greatly relies on the quality of data selected. For this particular project, the CICIDS 2017 (Canadian Institute for Cybersecurity Intrusion Detection System) will be employed. The CICIDS 2017 is one of the most popular datasets used in research in the network intrusion detection field. It includes realistic and labeled network traffic that was collected within five days in a controlled but realistic network environment.

The dataset consists of many network flows where the main label that will be used in this particular project is BENIGN. The label refers to normal network traffic. The data will consist of features that have been obtained from each individual network flow through the use of the CICFlowMeter program and the total number of features is seventy-eight.

The CICIDS 2017 data set will be used in this project because it simulates realistic network traffic behavior, provides publicly available data with labelling, and has been recognized within the cybersecurity field to be a credible basis for creating intrusion detection algorithms.

Data Preprocessing

Raw flow data may have noise, missing entries, infinity, and unimportant attributes that could affect the model's performance adversely. Hence, preprocessing is necessary before analyzing the dataset further.

Preprocessing encompasses the following steps:

- Getting rid of duplicate observations and observations that have null or infinite values. When you use CICFlowMeter to figure out the flow features, you often get this kind of data because of zero division errors.
- Getting rid of features that have no variance or very little variance because they don't help tell the difference between samples.
- Using Min-Max scaling to change all numeric features so that they are between 0 and 1. This stops one feature from being more important than others because of differences in scale.
- Encoding the label column, with traffic flows labeled as benign coded as 0, while attacks are coded as 1. This makes the problem a binary classification task.
- Using stratification to divide the data into a training set of 80% and a testing set of 20%

.Feature Engineering

The CICIDS 2017 dataset offers 78 extracted network flow features provided by the CICFlowMeter tool. They represent a wide variety of traffic attributes and constitute an essential input data for the machine learning model. Some of the main categories of the used features are the following:

- Flow level features including the flow duration, total number of forwarded and backwards packets, total bytes exchanged and bytes per second of the flow.
- Packet length-related features including mean, standard deviation, minimum and maximum length of forward and backward packets.
- Features related to the arrival of packets like mean and standard deviation of inter-packet arrival times.
- Features indicating the behavior of the connections based on the frequency of TCP flags like SYN, ACK, FIN, URG, and PSH.
- Header features including length-related statistics for headers and sizes of segments of forward and backward packets.

The importance of the features is determined by using the built-in Gini impurity criterion feature importance in the case of training the Random Forest classifier. The insignificant features are excluded during the post-training stage.

Model Training

After completing the construction of the feature matrix, the Machine Learning classifier is trained based on the labeled training data. For our project, the Random Forest classifier was chosen as the principal one because of its efficiency in dealing with tabular data, noise tolerance, and high dimensionality of the input.

Random Forest is an ensemble learning technique that creates many decision trees when it is trained. Each tree is trained on the randomly selected training data subsets, where features used for splitting are also selected randomly. Finally, voting takes place among all the trees to decide on the class prediction.:

$y_{pred} = mode \{ h_1(x), h_2(x), \dots, h_T(x) \}$ where $h_t(x)$ is the prediction of the t -th decision tree and T is the total number of trees in the forest. This aggregation reduces variance compared to a single decision tree and significantly improves generalisation performance.

The dataset is split into 80% training data and 20% test data using stratified sampling to maintain proportional class representation. The Random Forest model is trained on the preprocessed and normalized feature matrix using the scikit-learn implementation.

Model Evaluation

After the training process, the model is tested against the test data set. For this, we use the confusion matrix that contains four metrics: True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). Based on this data, we calculate the following parameters:

- Accuracy: A parameter indicating the degree of correctness of the model's decisions. $Accuracy = (TP + TN) / (TP + TN + FP + FN)$
- Precision: Parameter showing what percentage of the predicted attack flows is an attack. $Precision = TP / (TP + FP)$
- Recall: The parameter that shows the number of correctly recognized attacks out of all true attacks. $Recall = TP / (TP + FN)$
- F1-score: The harmonic mean between precision and recall.

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

The most critical value in the case under consideration is the recall parameter since the failure to detect an actual attack (a false negative) allows the attacker to continue their actions on the network undisturbed. The selection of the final model is carried out according to the weighted F1-score value. Moreover, a verification step through stratified cross-validation of five folds is made.

Model Optimisation

The optimization of the Random Forest model takes place through hyperparameter tuning using the grid search cross-validation technique. The critical hyperparameters that undergo optimization in this case are the number of decision trees ($n_estimators$), tree depth (max_depth), minimum sample size for node splitting ($min_samples_split$), and maximum number of features considered for splitting ($max_features$).

The problem of class imbalance between the BENIGN and attack classes is tackled using the `class_weight='balanced'` parameter for the Random Forest classifier, which gives more importance to the minority class.

Deployment and Real-Time Detection

After getting optimal results on the test set from the Random Forest algorithm, it is saved to the disk using the joblib library, which allows quick loading and prediction with the model. This way,

both the trained model and the preprocessing pipeline are deployed for network flows classification in real time.

Network flows are extracted continuously with the help of CICFlowMeter and then preprocessed using the same pipeline as used in the training stage. Finally, prediction is done using the trained model, and outputted predictions include binary classification — BENIGN or attack — and a confidence score that comes from class probabilities obtained by the Random Forest classifier.

Deployment process is not free of difficulties, but a system can solve at least some of them, e.g., speed and adaptivity. Feedback is provided constantly for the possibility of periodic training with new data..

Conclusion of Methodology

The above methodology is designed to offer a specific and practical way to detect network intrusions through the use of the CICIDS 2017 dataset as well as the Random Forest method. Through data preprocessing, feature extraction from CICFlowMeter, and model training/evaluation as well as an effective inference pipeline, this project aims to classify traffic accurately into benign classes.
